LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

04

Linux Package Management and Software Installation

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 What Is Package Management?
02 Linux Package Managers
03 APT (Ubuntu and Debian)
04 DNF and YUM (RPM-Based)
05 Package Repositories
06 Dependencies
07 Installing Package Files
08 Checking Installed Software
09 Versions, Holds and Rollback
10 Services After Installation
11 Troubleshooting
12 Preparing an Ubuntu Server
13 Interview Q&A
14 Takeaways and Exercises

Ravi Tomar
Linux · DevOps · Cloud Infrastructure

CHAPTER OBJECTIVE

Understand how Linux software is installed, updated, removed and maintained. Learn package managers, repositories, dependency management and practical troubleshooting commands used by DevOps engineers.

1 What Is Package Management?

In Linux, software is commonly distributed as packages. A package contains application files, configuration files, metadata and sometimes installation scripts.

Instead of manually downloading and installing every application, Linux provides package managers to automate software installation and maintenance.

For example, to install Nginx on Ubuntu:

```
sudo apt install nginx
```

The package manager automatically downloads the required package, checks dependencies and installs the software.

Why Package Management Is Important

- Install and remove software.
- Manage software versions.
- Install required dependencies.
- Apply security updates.
- Maintain consistent server environments.

2 Linux Package Managers

Different Linux distributions use different package management systems.

Distribution	Package manager	Package format
Ubuntu	APT	`.deb`
Debian	APT	`.deb`
Linux Mint	APT	`.deb`
RHEL	DNF / YUM	`.rpm`
Rocky Linux	DNF	`.rpm`
AlmaLinux	DNF	`.rpm`
Fedora	DNF	`.rpm`
Amazon Linux 2	YUM	`.rpm`
Amazon Linux 2023	DNF	`.rpm`

Two major package families are:

- **Debian-based:** Uses `.deb` packages and tools such as `apt` and `dpkg`.
- **RPM-based:** Uses `.rpm` packages and tools such as `dnf` and `rpm`.

Before using package commands, identify your Linux distribution:

```
cat /etc/os-release
```

INTERVIEW QUESTION

How are Fedora, CentOS Stream, RHEL, Rocky Linux and AlmaLinux related?

```
Fedora  →  CentOS Stream  →  RHEL  →  Rocky Linux / AlmaLinux
(newest)    (preview of RHEL)  (enterprise)   (free RHEL-compatible rebuilds)
```

Fedora introduces new features first. CentOS Stream shows what is coming in the next RHEL release. RHEL is the stable enterprise product. Rocky Linux and AlmaLinux are free community rebuilds compatible with RHEL.

3 APT – Package Management in Ubuntu and Debian

APT (Advanced Package Tool) is commonly used to manage software on Ubuntu and Debian systems.

Update Package Information

```
sudo apt update
```

This downloads the latest package information from configured repositories. It does not upgrade installed software.

Upgrade Installed Packages

```
# See which packages can be upgraded
apt list --upgradable

# Install available updates
sudo apt upgrade
```

Install Software

```
# Install a package
sudo apt install nginx

# Install multiple packages
sudo apt install nginx git curl
```

Remove Software

```
# Remove the package but keep its system configuration files
sudo apt remove nginx

# Remove the package and its system configuration files
sudo apt purge nginx

# Remove unused dependencies
sudo apt autoremove
```

Search and Inspect Packages

```
# Search for packages
apt search nginx

# Check package information
apt show nginx

# List installed packages
apt list --installed
```

Important Distinction

Command	Purpose
`apt update`	Refresh package information
`apt upgrade`	Upgrade installed packages without removing any
`apt full-upgrade`	Upgrade packages and remove or replace packages if required
`apt install`	Install software
`apt remove`	Remove software
`apt purge`	Remove software and system configuration
`apt autoremove`	Remove unused dependencies

INTERVIEW QUESTION

What is the difference between `apt` and `apt-get` ?

`apt` is a newer, user-friendly command with progress bars and cleaner output, designed for interactive use. `apt-get` is the older tool with a stable output format, so it is preferred in scripts and automation.

DEVOPS NOTE

On production servers, review available updates and test them before applying changes. Package upgrades can affect running applications and services.

4 DNF and YUM – Package Management in RPM-Based Systems

RPM-based distributions use the RPM package format.

DNF is the modern package manager used by current Fedora, RHEL-family systems and Amazon Linux 2023.

YUM is the older package management interface. On newer systems, `yum` may act as a compatibility command backed by DNF.

```
# Check for and install updates
sudo dnf check-update
sudo dnf upgrade

# Install software
sudo dnf install nginx

# Remove software
sudo dnf remove nginx

# Search for packages
dnf search nginx

# Display package information
dnf info nginx

# List installed packages
dnf list installed

# Clean package cache
sudo dnf clean all
```

Common DNF Commands

Command	Purpose
`dnf install`	Install software
`dnf remove`	Remove software
`dnf upgrade`	Upgrade installed packages
`dnf search`	Search packages
`dnf info`	Display package information
`dnf list installed`	List installed packages
`dnf clean all`	Clean cached package data

5 Understanding Package Repositories

A repository is a location where Linux packages and their metadata are stored.

When you run `sudo apt install nginx`, APT searches the configured repositories, downloads the required package and installs it.

Repositories may be:

- Official distribution repositories.
- Third-party software repositories.
- Internal company repositories.

APT Repository Configuration

Ubuntu and Debian commonly store repository configuration in:

```
/etc/apt/sources.list
/etc/apt/sources.list.d/
```

```
# View configured repositories and their priorities
apt-cache policy
```

NOTE

Ubuntu 24.04 and newer use a new repository format stored in files such as `/etc/apt/sources.list.d/ubuntu.sources`. `apt-cache policy` works with both the old and new formats.

DNF Repository Configuration

RPM-based systems commonly store repository files in:

```
/etc/yum.repos.d/
```

```
# List enabled repositories
sudo dnf repolist
```

Why Repositories Use GPG Keys

Repositories sign their packages with a GPG key. The package manager checks this signature before installing, which confirms that the package really came from the trusted source and was not modified.

Adding a Third-Party Repository (Ubuntu)

```
# 1. Download the repository's GPG key
curl -fsSL https://example.com/repo.gpg | sudo gpg --dearmor -o /etc/apt/keyrings/example

# 2. Add the repository, linked to that key
echo "deb [signed-by=/etc/apt/keyrings/example.gpg] https://example.com/apt stable main"

# 3. Refresh package information
sudo apt update
```

INTERVIEW NOTE

The old `apt-key add` method is deprecated. Modern systems store each repository's key separately and link it using `signed-by`.

Adding a Repository (RPM-Based)

```
# Add a repository from a URL
sudo dnf config-manager --add-repo https://example.com/example.repo

# Enable EPEL (Extra Packages for Enterprise Linux) on Rocky / AlmaLinux
sudo dnf install epel-release
```

EPEL provides many extra packages, such as `htop`, that are not included in the default RHEL-family repositories.

> **DEVOPS NOTE**
>
> In production environments, organizations may use internal repositories to control which software versions servers can install. This improves consistency and helps manage software updates.

6 Understanding Dependencies

A dependency is another package or library required by an application to work correctly. For example, an application may require:

- A specific runtime.
- Shared libraries.
- Supporting tools.
- Other system packages.

Package managers automatically resolve and install many required dependencies.

```
# Check dependencies of a Debian package
apt-cache depends nginx

# Check package requirements on an RPM-based system
dnf repoquery --requires nginx
```

What Happens When Dependencies Are Missing?

You may encounter errors such as:

```
Package dependency problems
Unmet dependencies
Missing required libraries
```

On Ubuntu, try:

```
sudo apt --fix-broken install
```

This attempts to fix broken package dependencies. Review the proposed changes before confirming.

7 Installing Software Using Package Files

Sometimes software is provided as a package file instead of being installed directly from a repository.

Installing a .deb Package

```
# Recommended: resolves dependencies
sudo apt install ./package.deb

# Alternative: low-level tool
sudo dpkg -i package.deb

# If dpkg reports dependency problems
sudo apt --fix-broken install
```

Installing an .rpm Package

```
# Recommended: resolves dependencies
sudo dnf install ./package.rpm

# Alternative: low-level tool
sudo rpm -ivh package.rpm
```

APT vs DPKG and DNF vs RPM

Tool	Purpose
APT	High-level Debian package management and dependency resolution
DPKG	Low-level installation and management of `.deb` packages
DNF	High-level RPM package management and dependency resolution
RPM	Low-level installation and management of `.rpm` packages

> **IMPORTANT**
>
> Prefer APT or DNF for normal installations because they handle repository access and dependency resolution.

8 Checking Installed Software and Versions

As a DevOps engineer, you frequently need to verify which software and versions are installed on a server.

```
# Application version commands
nginx -v
git --version
python3 --version

# Installed package version on Ubuntu
dpkg -l nginx

# Installed package version on an RPM-based system
rpm -q nginx

# Find the executable path
which nginx
command -v nginx
```

`command -v` is a portable shell built-in that also recognizes aliases and built-ins, which is why it is preferred in scripts.

Which Package Owns a File?

```
# Ubuntu / Debian
dpkg -S /usr/sbin/nginx

# RPM-based systems
rpm -qf /usr/sbin/nginx
```

Which Files Did a Package Install?

```
# Ubuntu / Debian
dpkg -L nginx

# RPM-based systems
rpm -ql nginx
```

DEVOPS EXAMPLE

If you find an unknown binary or configuration file on a server, `dpkg -S` or `rpm -qf` tells you which package installed it.

9 Managing Versions, Holds and Rollback

In production, you often need exact control over which version is installed, and a way to stop critical packages from upgrading unexpectedly.

Install a Specific Version

```
# Ubuntu: list available versions
apt list -a nginx

# Ubuntu: install a specific version
sudo apt install nginx=<version>

# RPM-based: list available versions
dnf list --showduplicates nginx

# RPM-based: install a specific version
sudo dnf install nginx-<version>
```

Prevent a Package from Upgrading

```
# Ubuntu: hold a package
sudo apt-mark hold nginx
apt-mark showhold
sudo apt-mark unhold nginx

# RPM-based: lock a version (requires the versionlock plugin)
sudo dnf install python3-dnf-plugin-versionlock
sudo dnf versionlock add nginx
dnf versionlock list
sudo dnf versionlock delete nginx
```

DEVOPS EXAMPLE

On Kubernetes nodes, `kubelet`, `kubeadm` and `kubectl` are commonly held with `apt-mark hold`. This prevents a routine `apt upgrade` from upgrading cluster components unexpectedly.

Package History and Rollback

```
# RPM-based: view package transaction history
dnf history

# View details of a transaction
dnf history info <ID>

# Undo a transaction
sudo dnf history undo <ID>
```

APT has no built-in undo command. On Ubuntu, check `/var/log/apt/history.log` and `/var/log/dpkg.log` to see what changed, then reinstall the previous version manually.

Security Updates

```
# RPM-based: install only security updates
sudo dnf upgrade --security
```

On Ubuntu, the `unattended-upgrades` service can install security updates automatically.

> INTERVIEW QUESTION
>
> **How do you prevent a package from being upgraded?**
>
> On Ubuntu, use `apt-mark hold package`. On RPM-based systems, use `dnf versionlock add package`.

10 Managing Services After Installation

Installing a package does not always mean its service is running. For example, after installing Nginx:

```
# Check its status
sudo systemctl status nginx

# Start the service
sudo systemctl start nginx

# Stop the service
sudo systemctl stop nginx

# Restart the service
sudo systemctl restart nginx

# Enable the service at boot
sudo systemctl enable nginx

# Enable and start it immediately
sudo systemctl enable --now nginx
```

IMPORTANT

Package management and service management are different tasks. A package manager installs software; `systemctl` manages services on systems using systemd.

Service management will be covered in detail in Chapter 5.

11 Package Management Troubleshooting

Scenario 1 Unable to locate package

```
E: Unable to locate package nginx
```

Possible reasons:

- Package information is outdated.
- Required repository is not configured.
- Package name is incorrect.
- Network or repository connectivity is unavailable.

```
sudo apt update
sudo apt install nginx
```

Scenario 2 Another package manager process is running

```
Could not get lock /var/lib/dpkg/lock-frontend
```

Another APT or DPKG process is currently installing or updating packages.

```
ps aux | grep -E 'apt|dpkg'
```

On a newly launched cloud server, this is often the `unattended-upgrades` service applying security updates in the background.

Wait for the active operation to finish. Do not delete lock files blindly, because doing so can corrupt package management state.

Scenario 3 Package installation is interrupted

Check and repair package configuration:

```
sudo dpkg --configure -a
sudo apt --fix-broken install
```

Scenario 4 Repository connection failure

```
# Check network connectivity
ping -c 4 8.8.8.8

# Check DNS resolution
getent hosts archive.ubuntu.com

# Test HTTP access to the repository
curl -I http://archive.ubuntu.com
```

In cloud environments, ping (ICMP) may be blocked by security rules even when the network works. Testing with `curl` is more reliable.

Also verify repository configuration and proxy settings if your environment uses them.

Scenario 5 GPG key error

```
NO_PUBKEY ... The following signatures couldn't be verified
```

The repository's signing key is missing or has expired. Download the current key from the software vendor's official instructions, link it using `signed-by`, then run `sudo apt update` again.

Never disable signature checking to work around this error.

12 Practical DevOps Example: Preparing an Ubuntu Server

Suppose you have launched a new Ubuntu EC2 instance and need to prepare it for application deployment.

Step 1 Update package information

```
sudo apt update
```

Step 2 Upgrade installed packages

```
sudo apt upgrade
```

Step 3 Install required tools

```
sudo apt install -y nginx git curl unzip
```

Step 4 Verify installation

```
nginx -v
git --version
curl --version
```

Step 5 Enable Nginx

```
sudo systemctl enable --now nginx
```

Step 6 **Verify service status**

```
sudo systemctl status nginx
```

This is a common initial server preparation workflow before deploying an application.

AUTOMATION TIP

In automation scripts and EC2 user data, package installation must never wait for user input. Use `-y` and disable interactive prompts:

```
export DEBIAN_FRONTEND=noninteractive
sudo apt-get update
sudo apt-get install -y nginx git curl unzip
```

13 Common Interview Questions

Q1. What is a Linux package manager?

A package manager installs, updates, removes and manages software packages and their dependencies.

Q2. What is the difference between APT and DNF?

APT is commonly used on Debian-based distributions, while DNF is used on RPM-based distributions.

Q3. What is the difference between `apt update` and `apt upgrade`?

`apt update` refreshes package information. `apt upgrade` installs available updates for installed packages.

Q4. What is the difference between APT and DPKG?

APT is a high-level package manager that handles repositories and dependencies. DPKG is a lower-level tool for managing `.deb` packages.

Q5. What is a Linux repository?

A repository is a package source containing software packages and metadata that a package manager can use.

Q6. What are package dependencies?

Dependencies are additional packages or libraries required for software to function correctly.

Q7. How do you install Nginx on Ubuntu?

```
sudo apt update
sudo apt install nginx
```

Q8. How do you install Nginx on Amazon Linux 2023?

```
sudo dnf install nginx
```

Q9. How do you check the installed version of a package?

On Ubuntu, use `dpkg -l package-name`. On RPM-based systems, use `rpm -q package-name`. You can also use the application's own version command when available.

Q10. What is the difference between `apt remove` and `apt purge`?

`apt remove` removes the package but generally leaves system configuration files. `apt purge` also removes those system configuration files.

Q11. Why should production servers not be upgraded without testing?

Updates can introduce compatibility issues, change dependencies or affect application behavior. Test updates before applying them to production.

Q12. What would you do if package installation failed because of dependency issues?

On Ubuntu, inspect the error and try `sudo apt --fix-broken install`. On RPM-based systems, inspect the DNF error and resolve the missing or conflicting dependencies.

Q13. How do you prevent a package from being upgraded?

Use `apt-mark hold package` on Ubuntu, or `dnf versionlock add package` on RPM-based systems.

Q14. How do you find which package a file belongs to?

Use `dpkg -S /path/to/file` on Ubuntu, or `rpm -qf /path/to/file` on RPM-based systems.

Q15. How do you roll back a package update on an RPM-based system?

Use `dnf history` to find the transaction ID, then run `sudo dnf history undo <ID>`.

14 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ What Linux package management is.
- ✓ APT and DNF package managers.
- ✓ Debian and RPM package formats.
- ✓ Installing, updating and removing software.
- ✓ Linux package repositories and GPG keys.
- ✓ Package dependencies.
- ✓ Installing `.deb` and `.rpm` files.
- ✓ Checking installed software, versions and file ownership.
- ✓ Installing specific versions, holding packages and rolling back updates.
- ✓ Managing services after installation.
- ✓ Troubleshooting common package management errors.

PRACTICAL EXERCISES

1. Identify your Linux distribution using `/etc/os-release`.
2. Update the package information.
3. Install Nginx, Git and curl.
4. Verify their installed versions.
5. Check the installed Nginx package details.
6. Check which repositories are configured.
7. Start and enable the Nginx service.
8. Remove a test package and inspect the difference between `remove` and `purge`.
9. Search for a package using APT or DNF.
10. Investigate a simulated package dependency error.
11. Hold a package using `apt-mark hold` and confirm it with `apt-mark showhold`.
12. Find which package owns `/usr/bin/curl`.

NEXT CHAPTER

Chapter 05: Process and Service Management

You will learn how Linux runs applications and background processes, how to monitor CPU and memory usage, manage services using systemd, and troubleshoot processes that become unresponsive or consume excessive resources.

END OF CHAPTER 4